ECPS204 Project2-Multi-threading canny edge

CHENG-CHIH, LEE, 29329351
JONATHAN KIM, 42411903

Project Description (2 pts)

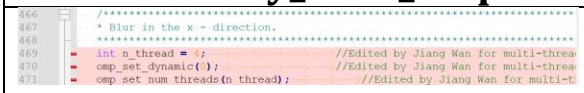
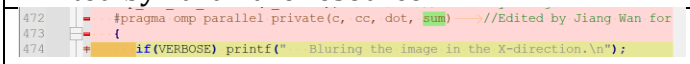
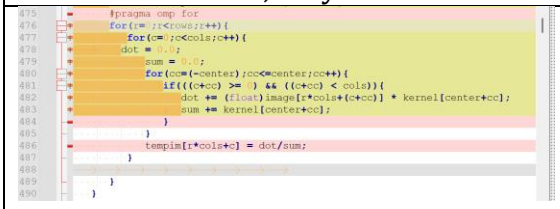
- The purpose of this project is to implement multithreading on the canny application. OpenMP and pthread are explored and implemented.
- The improvement of performance is expected to be measured and discussed.

1. Experimental Setup (4 pts)

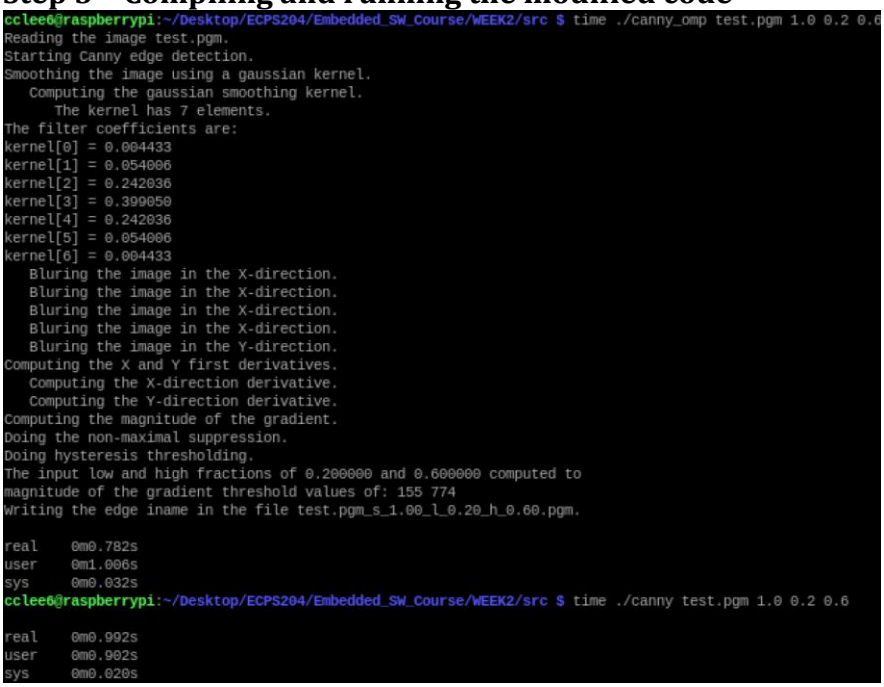
- For this project, we set up a Git repository to exchange files between the RPi and laptop. Following the instructions, we get each step done and recorded.
- The local canny file is used to study multithreading method. And the developed canny.cpp in proj1 is also modified to implement multithreading.

2. Results (6 pts)

Step 2 – Understanding the modifications on canny_local.c with OpenMP

canny_local_omp.c	canny_local_.c
 In Blur_x, omp is implemented to create 4 threads. omp_set_dynamic(0) configured omp to not changing thread numbers dynamically unless limited by hardware resource.	 None
 Parallel variable copies are created for each thread. Otherwise, they will be shared.	None
 #pragma omp for… dertermins the for loop to be run separately in each thread. “r” is copied here,	None

Step 3 – Compiling and running the modified code



#canny_local_omp

real 0m0.782s
user 0m1.006s
sys 0m0.032s

#canny_local

real 0m0.992s
user 0m0.902s
sys 0m0.020s

From the results, we can see with multithreading implementation, the real run time is shortened. However, the user and system time increased. The real time is the wall time; the user time is the total cpu time and system time is the kernel time. With multithreading, the wall time is improved and cpu/system time increased due to multithread/core execution.

Step 4 – Use OpenMP to parallelize Blur in the y direction

```
492     /*******************************************************************************
493     * Blur in the y - direction.
494     *******************************************************************************/
495     omp_set_dynamic(0);	//Edited by CHENG-CHIH LEE for multi-thread
496     omp_set_num_threads(n_thread);	//Edited by CHENG-CHIH LEE for multi-thread
497     #pragma omp parallel private(r, rr, dot, sum)	//Edited by CHENG-CHIH LEE for multi-thread
498     {
499         if(VERBOSE) printf("   Bluring the image in the Y-direction.\n");
500         #pragma omp for
501         for(c=0;c<cols;c++){
502             for(r=0;r<rows;r++){
503                 sum = 0.0;
504                 dot = 0.0;
505                 for(rr=(-center);rr<=center;rr++){
506                     if(((r+rr) >= 0) && ((r+rr) < rows)){
507                         dot += tempim[(r+rr)*cols+c] * kernel[center+rr];
508                         sum += kernel[center+rr];
509                     }
510                 }
511                 (*smoothedim)[r*cols+c] = (short int)(dot*BOOSTBLURFACTOR/sum + 0.5);
512             }
513         }
514     }
515     free(tempim);
516     free(kernel);
517 }
```

OpenMP is implemented for Blur Y shown as above. In BlurY multithreading implementation, n_thread isn't declared again since it was done before BlurX. The variables to be copied are (r, rr, dot, sum).
tempim and kernel are freed after all the threads are done execution.

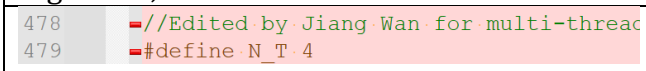
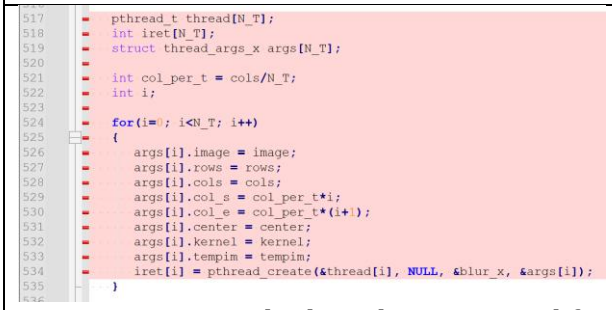
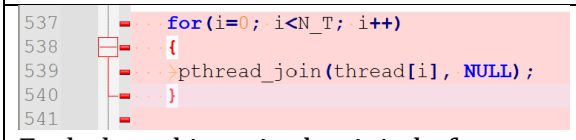
```
cclee6@raspberrypi:~/Desktop/ECPS204/Embedded_SW_Course/WEEK2/src $ time ./canny_omp test.pgm 1.0 0.2 0.6
Reading the image test.pgm.
Starting Canny edge detection.
Smoothing the image using a gaussian kernel.
   Computing the gaussian smoothing kernel.
      The kernel has 7 elements.
The filter coefficients are:
kernel[0] = 0.004433
kernel[1] = 0.054006
kernel[2] = 0.242036
kernel[3] = 0.399050
kernel[4] = 0.242036
kernel[5] = 0.054006
kernel[6] = 0.004433
   Bluring the image in the X-direction.
   Bluring the image in the X-direction.
   Bluring the image in the X-direction.
   Bluring the image in the X-direction.
   Bluring the image in the Y-direction.
   Bluring the image in the Y-direction.
   Bluring the image in the Y-direction.
   Bluring the image in the Y-direction.
Computing the X and Y first derivatives.
   Computing the X-direction derivative.
   Computing the Y-direction derivative.
Computing the magnitude of the gradient.
Doing the non-maximal suppression.
Doing hysteresis thresholding.
The input low and high fractions of 0.200000 and 0.600000 computed to
magnitude of the gradient threshold values of: 155 774
Writing the edge iname in the file test.pgm_s_1.00_l_0.20_h_0.60.pgm.

real    0m0.662s
user    0m0.913s
sys     0m0.044s
```

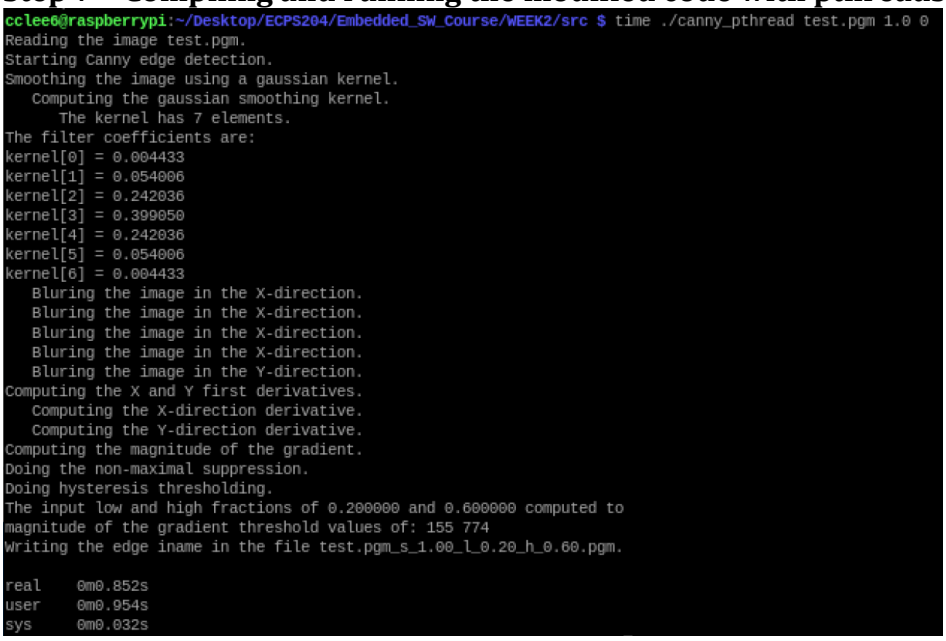
From this snapshot, we can see there are also 4 threads created for Blur Y.
And from the time result below, we can tell the real time is improved by approximately **33%** compared to the original application without multithreading and **15 %** compared to the application which only implemented multithreading in BlurX.

real 0m0.662s
user 0m0.913s
sys 0m0.044s

Step 6 – Understanding the modifications on canny_local.c with pthread

canny_local_pthread.c	canny_local_.c
 Structure is created for passing arguments for multithreading	None
 blur_x is modified to take argument and have local variables declared with init value from the argument,	None
 Number of thread is defined as 4	None
 In gaussian smooth, threads are created for BlurX and executed	None
 Each thread is waited to join before process moves on.	None

Step 7 – Compiling and running the modified code with pthreads



#canny_pthread
real 0m0.852s
user 0m0.954s
sys 0m0.032s
#canny_local_omp
real 0m0.782s
user 0m1.006s
sys 0m0.032s
#canny_local
real 0m0.992s
user 0m0.902s
sys 0m0.020s

From the results, we can tell the implementation of pthread did improve the wall time, but not as much as OpenMP.

Step 8 – Modifying the Blur in the y - direction with pthreads

```
struct thread_args_x
{
    unsigned char *image;
    int rows;
    int cols;
    int col_s;
    int col_e;
    int row_s;
    int row_e;
    int center;
    float *kernel;
    float *tempim;
    short int *smoothedim;
};
```

Arguments added for Blur Y use.

```
//Added by CHENH-CHIH LEE for multi-thread
void *blur_y(void *arguments)
{
    /*******************************************************************************
    * Blur in the y - direction.
    *******************************************************************************/
    struct thread_args_x *args = arguments;

    unsigned char *image = args->image;
    int rows = args->rows;
    int cols = args->cols;
    int row_s = args->row_s;
    int row_e = args->row_e;
    int center = args->center;
    float *kernel = args->kernel;
    float *tempim = args->tempim;
    short int *smoothedim = args->smoothedim;

    int r, c, rr;
    float dot, sum;

    if(VERBOSE) printf("   Bluring the image in the Y-direction.\n");
    for(r=row_s;r<row_e;r++){
        for(c=0;c<cols;c++){
            sum = 0.0;
            dot = 0.0;
            for(rr=(-center);rr<=center;rr++){
                if(((r+rr) >= 0) && ((r+rr) < rows)){
                    dot += tempim[(r+rr)*cols+c] * kernel[center+rr];
                    sum += kernel[center+rr];
                }
            }
            (smoothedim)[r*cols+c] = (short int)(dot*BOOSTBLURFACTOR/sum + 0.5);
        }
    }
    return NULL;
}
```

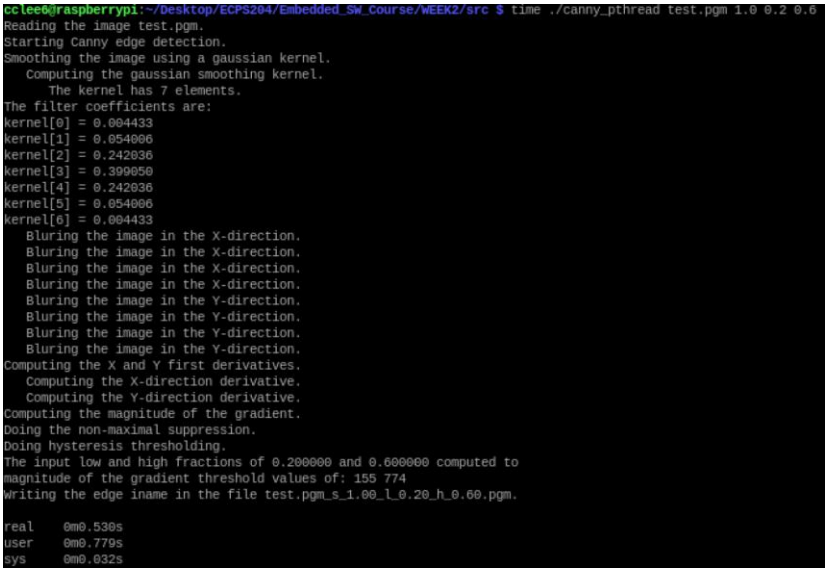
Blur Y seperated for multithreading. Iteration on rows, so we modified the for loops as cols in the inner loop.

```
    for(i=0; i<N_T; i++)
    {
    pthread_join(thread[i], NULL);
    }
    //Thread creation for Blur Y
    int row_per_t = rows/N_T;

    for(i=0; i<N_T; i++)
    {
        args[i].image = image;
        args[i].rows = rows;
        args[i].cols = cols;
        args[i].row_s = row_per_t*i;
        args[i].row_e = row_per_t*(i+1);
        args[i].center = center;
        args[i].kernel = kernel;
        args[i].tempim = tempim;
        args[i].smoothedim = *smoothedim;
        iret[i] = pthread_create(&thread[i], NULL, &blur_y, &args[i]);
    }

    for(i=0; i<N_T; i++)
    {
    pthread_join(thread[i], NULL);
    }
```

Threads created for Blur Y.



Now from the snapshot, we can see there are 4 threads created for Blur Y.
Time results are below:

#canny_pthread X+Y

real 0m0.530s
user 0m0.779s
sys 0m0.032s

#canny_local_omp

real 0m0.662s
user 0m0.913s
sys 0m0.044s

#canny_local

real 0m0.992s
user 0m0.902s
sys 0m0.020s

We can see the wall time is improved and having better performance than omp.

Step 9 – Comparing OpenMP and pthreads

	Original	OMP	OMP to Original	pThread	pThread to Original
real	0m0.992s	0m0.662s	Improve 33.3%	0m0.530s	Improve 46.6%
user	0m0.902s	0m0.913s	Deteriorate 1.2%	0m0.779s	Improve 13.6%
sys	0m0.020s	0m0.044s	Deteriorate 120%	0m0.032s	Deteriorate 60%

From the comparison, we can see that OpenMP is bringing large overhead to the kernel loading. And pthread is also resulting in better WALL time improvement.

Step 10 - Applying multi-threading (in OpenMP and pthread individually) to the camera-capturing-and-edge-detection system (in Proj #1-b)

#150 frames were taken in both OMP and pthread implementations.

1. Implementing OpenMP

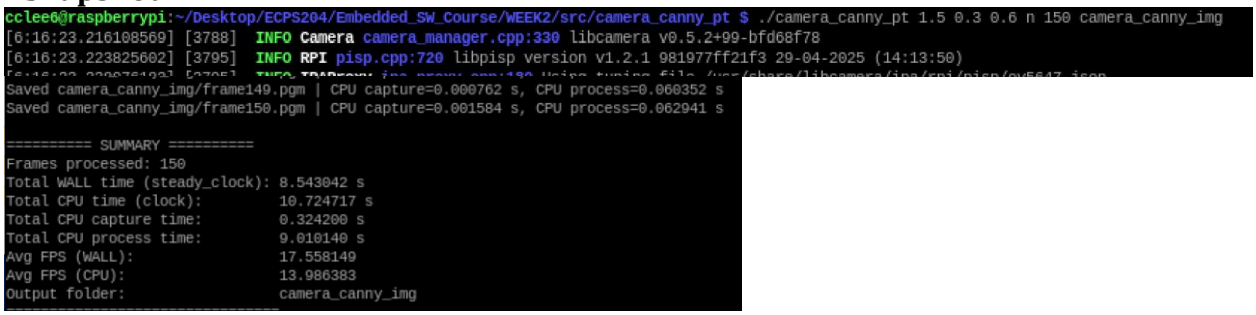
i. Snapshot

ii. Performance

========== SUMMARY ==========
Frames processed: 150
Total WALL time (steady_clock): 9.237100 s
Total CPU time (clock): 14.016798 s
Total CPU capture time: 0.315001 s
Total CPU process time: 12.469819 s
Avg FPS (WALL): 16.238863
Avg FPS (CPU): 10.701446
Output folder: camera_canny_img
================================

2. Implementing pthread

i. Snapshot

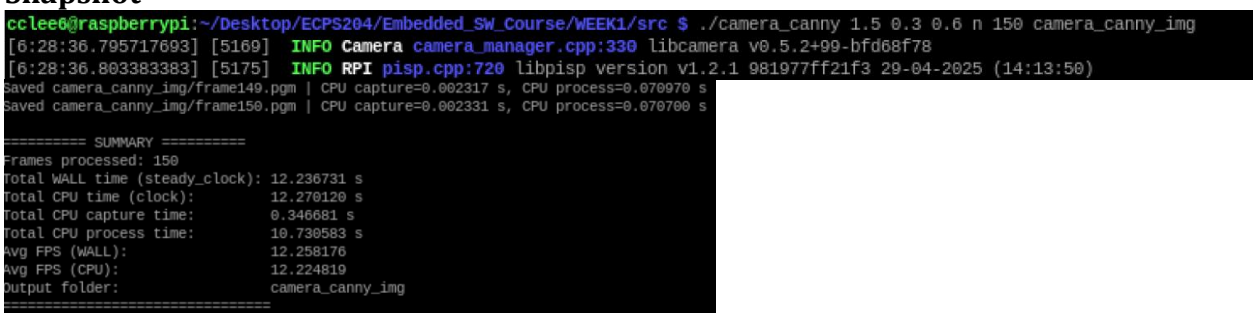


ii. Performance

========== SUMMARY ==========
Frames processed: 150
Total WALL time (steady_clock): 8.543042 s
Total CPU time (clock): 10.724717 s
Total CPU capture time: 0.324200 s
Total CPU process time: 9.010140 s
Avg FPS (WALL): 17.558149
Avg FPS (CPU): 13.986383
Output folder: camera_canny_img
================================

3. Implementation from Proj1

i. Snapshot



ii. Performance

========== SUMMARY ==========
Frames processed: 150
Total WALL time (steady_clock): 12.236731 s
Total CPU time (clock): 12.270120 s
Total CPU capture time: 0.346681 s
Total CPU process time: 10.730583 s
Avg FPS (WALL): 12.258176
Avg FPS (CPU): 12.224819
Output folder: camera_canny_img
==============================

4. Performance Comparison

	Proj1	OMP	Impv. OMP to Proj1	pThread	Impv. pThread to Proj1
Total WALL time	12.236731 s	9.237100 s	Impv. 24.5%	8.543042 s	Impv. 30.2%
Total CPU time (clock)	12.270120 s	14.016798 s	Detero. 14.2%	10.724717 s	Impv. 12.6%
Total CPU capture time	0.346681 s	0.315001 s	Impv. 9.1%	0.324200 s	Impv. 6.5%
Total CPU process time	10.730583 s	12.469819 s	Detero. 16.2%	9.010140 s	Impv. 16.1%
Avg FPS (WALL)	12.258176	16.238863	Impv. 32.5%	17.558149	Impv. 43.3%
Avg FPS (CPU)	12.224819	10.701446	Detero. 12.5%	13.986383	Impv. 14.4%

From our implementation and testing, we noticed that OMP performs worse than pthread, which is quite identical in the previous “local” single image application.
However, we get wall time improvement in both implementations.

3. Problems and Discussion (6 pts)

- F
-

4. Conclusion (2 pts)

- A
- .